

Introduction

Course Code: CSE207

Antu Chowdhury
Lecturer,
Dept. of CSE

Course Information

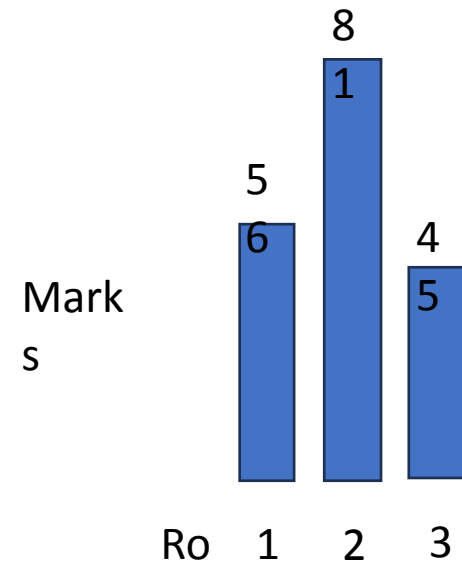
Component	Marks (%)
CT/Quiz	10%
Mid Term Exam	30%
Final Exam	30%
Laboratory Performance and Lab VIVA	10%
Lab Final	10%
Mini Project	10%
Total	100%

Data Structures

- **Data** refers to raw, unorganized facts that **do not have meaning on their own**. These can be numbers, characters, symbols, images, or sounds that are collected but not yet processed
- **Information** is data that has been **processed, organized, and given meaning**. It is useful for decision-making or understanding.

Student Roll	Marks
1	56
2	81
3	45

Data



Information

Data Structures

- **Program → Set of instructions → Performs operations on data**
When a program deals with data, the way that data is organized in the main memory during execution is called a **data structure**.

Need of Data Structures

- The choice of a good data structure makes it possible to perform a variety of critical operations effectively. An efficient data structure also uses minimum memory space and execution time to process the structure.
- A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data. There are different basic and advanced types of data structures that are used in almost every program or software system that has been developed. So we must have good knowledge of data structures.

Algorithms

- An algorithm is a step-by-step set of instructions for solving a specific problem or performing a task.
 - Systematic sequence of actions designed to achieve a particular goal or function
 - Program = Data Structure + Algorithm
- Information is manipulated by systematic and step by step procedure called **Algorithm**
- Manipulation means adding, deleting, searching, rearranging data items

Data types: Classification

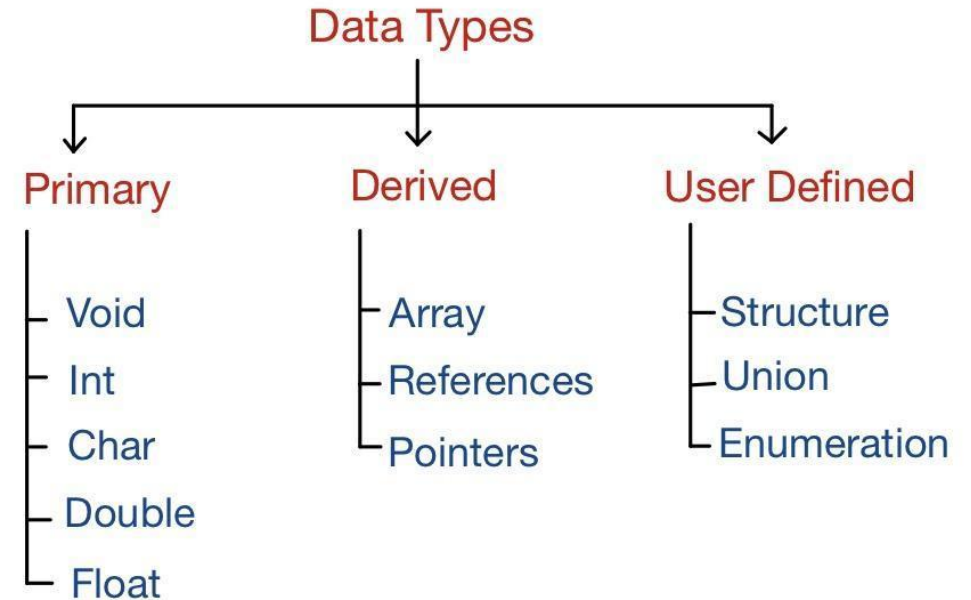
Two important things about data types:

1. Defines a certain domain of values.
2. Defines Operations allowed on those values.

EXAMPLE:

int type

- Takes only integer values
- Operations: addition, Subtraction, multiplication, bitwise operations etc.



1. Basic (Primitive) Data Types

Data Type	Description	Example	Size
int	Integer numbers	int x = 5;	4 bytes
float	Decimal numbers (single precision)	float y = 3.14;	4 bytes
double	Decimal (double precision)	double z = 3.14159;	8 bytes
char	Single character	char ch = 'A';	1 byte
void	No value (used for functions)	void print();	0 bytes

2. Derived Data Types

These are based on basic types:

Type	Description	Example
Array	Collection of same type elements	<code>int a[5];</code>
Pointer	Stores address of another variable	<code>int *p = &a;</code>
Function	A block that returns a type	<code>int sum();</code>

3. User-Defined Data Types

- The operations and values of user defined data types are not specified in the language itself but is specified by the user.
- By using structures, we are defining our own type by combining other data types.

Type	Description	Example
struct	Groups multiple variables	struct Student { int id; };
union	Memory shared by all members	union Data { int i; float f; };

Modifiers

Used to change size and behavior of basic types.

Modifier	Usage	Example
signed	Allows negative and positive	signed int x;
unsigned	Only positive	unsigned int x;
short	Smaller range	short int x;
long	Larger range	long int x;

Abstract Data Types (ADT)

ADTS are like user defined data types which defines operations on values using functions without specifying what is there inside the function and how the operations are performed.

EXAMPLE:

Stack ADT

A stack consists of elements of the same type arranged in a sequential order.

Operations:

initialize()-initializing it to be empty

Push()-Insert an element into the stack

Pop() Delete an element from the stack

isEmpty() -checks if stack is empty

isFull() -checks if stack is full

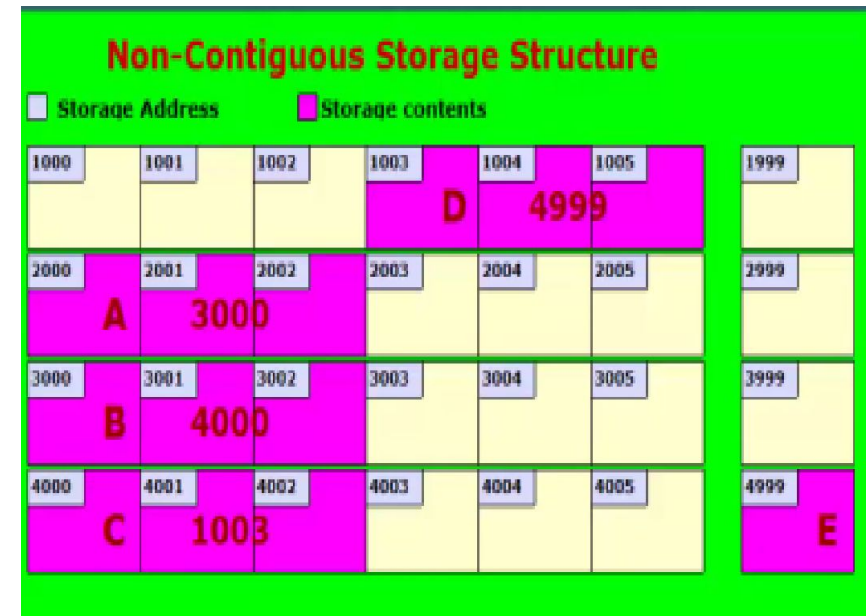
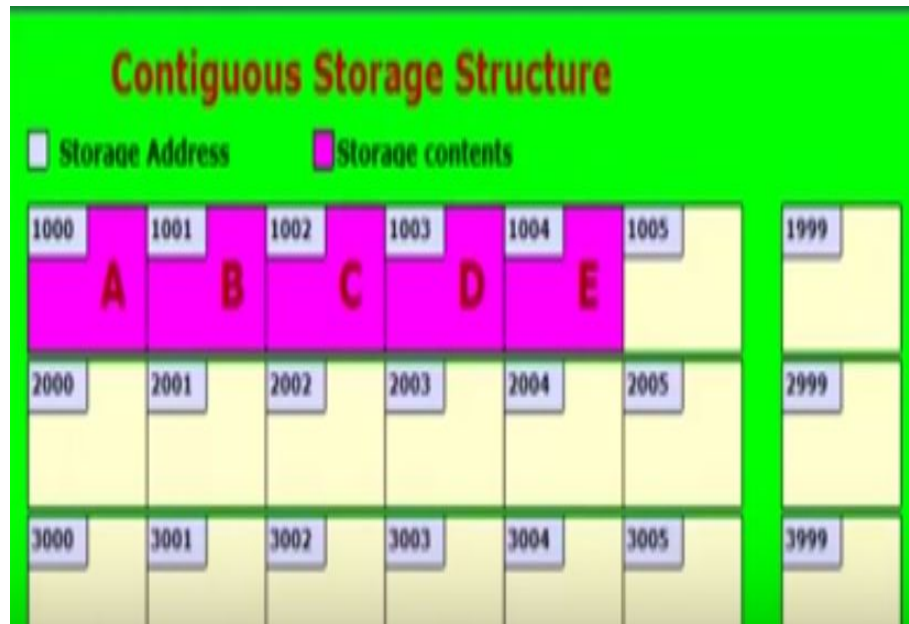
There are multiple ways to implement an ADT.

EXAMPLE:

A stack ADT can be implemented using arrays or linked lists.

Storage Structures

- It describes how the data item are stored in the memory
- Provides a map how the data elements are allocated memory space
- Elements are stores in consecutive and disjoint storage of memory location
- Each location is identified by unique number called memory address



Stack and Heap

- **memory is divided** into different sections during program execution. Two of the most important are:

- **Stack Memory**



Features:

- Memory is allocated automatically
- Follows LIFO (Last-In, First-Out) structure
- Stores:
 - Local variable
 - Function call data (parameters, return address)
- Fast allocation and deallocation

- **Heap Memory**

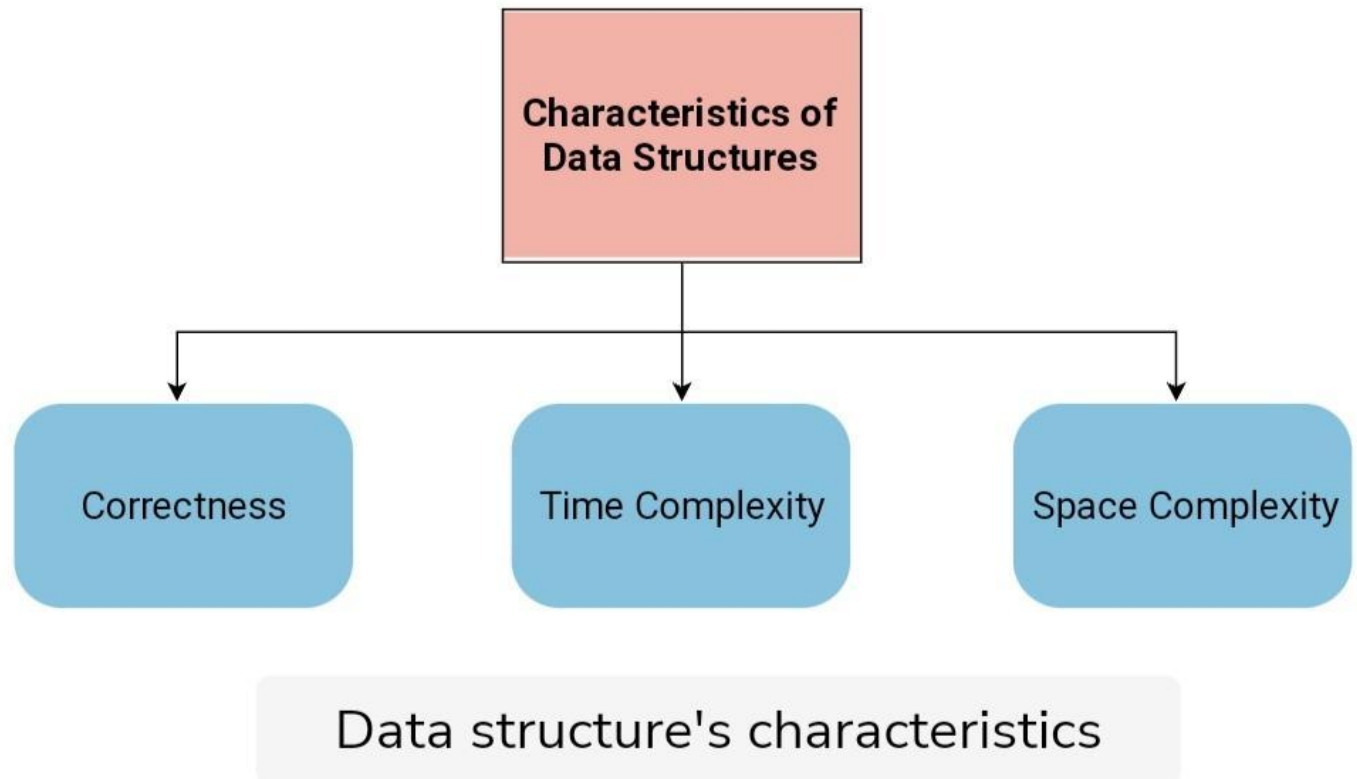


Features:

- Manually managed memory
- Used for:
 - Dynamic allocation (malloc, calloc)
 - Data structures like linked lists, trees
- Slower than stack

Characteristics

- Correctness
- Time Complexity
- Space Complexity



Correctness

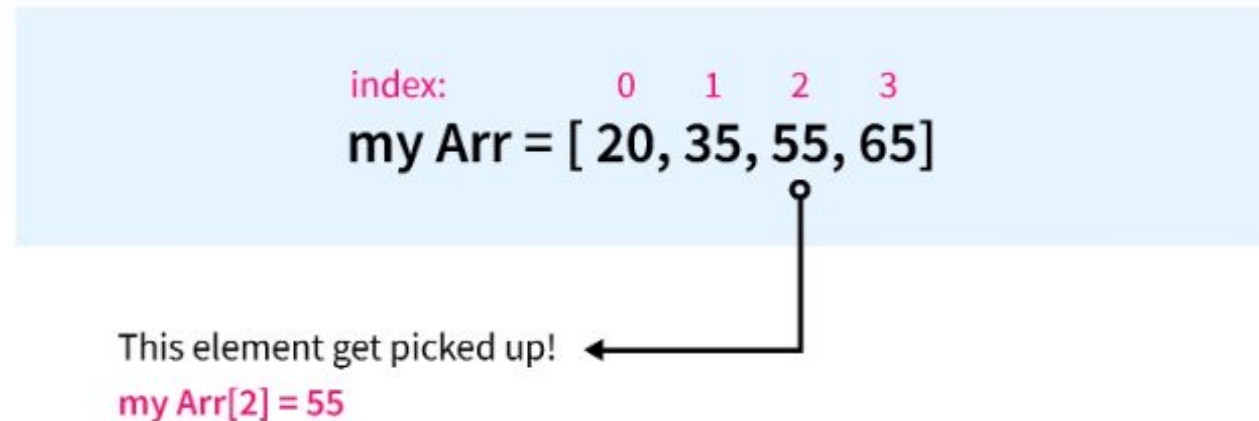
- Correctness here means that the algorithm always produces the expected output.
- It follows the ground truth for the **range of valid inputs**, and eventually, **it terminates**.
- It is important because you're relying on it for the desired output.

Time Complexity

- The execution time of a data structure as a function of the length of the input is known as time complexity.
- It's a type of computation complexity that describe the time your algorithm needs to be executed.
- The duration it takes for a computer program to solve a certain problem based on the size of the problem

Time Complexity

- It should be *as small as possible* because the more time your data structure takes, it will decrease its efficiency.
- Example: The access time complexity of an array is $O(1)$
- This is because you can **directly access** any element of the array using its index.



Space Complexity

- The **total memory space** your data structure needs for its execution or to work properly is known as space complexity.
- Again, it should be *as small as possible* for data structures because it's one of the important factors in efficiency.
- Example: A one dimensional array with n elements has a space complexity of $O(n)$

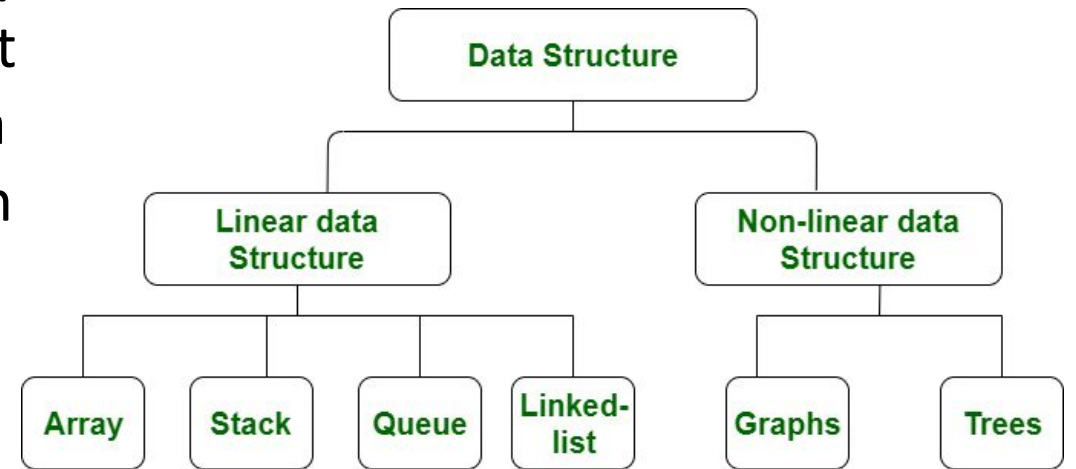
Data structure Types

Linear Data Structure:

Data structure where data elements are arranged sequentially or linearly where each and every element is attached to its previous and next adjacent is called a **linear data structure**. In linear data structure, single level is involved. Therefore, we can traverse all the elements in single run only.

Non-linear Data Structure:

Data structures where data elements are not arranged sequentially or linearly are called non-linear data structures. In a non-linear data structure, single level is not involved. Therefore, we can't traverse all the elements in single run only.

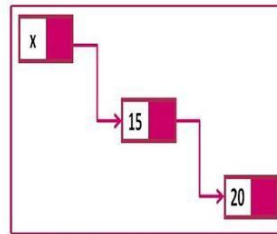


Data Structure

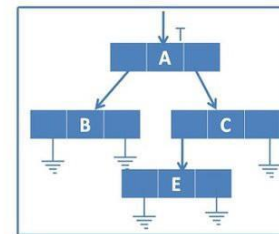
- Data Structures



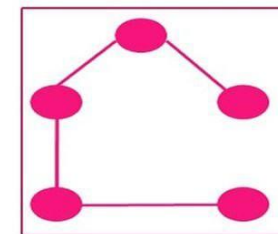
Sorting



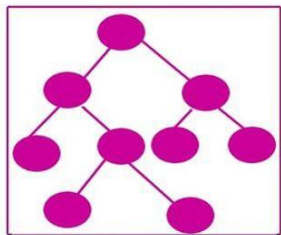
Link list



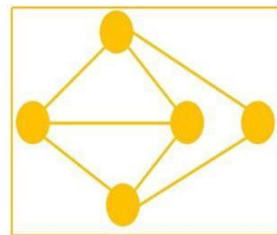
list



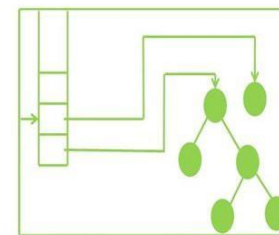
spanning tree



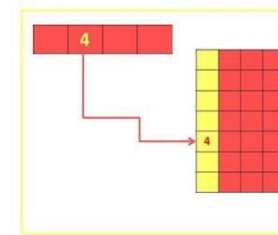
Tree



Graph



Stack



Hashing

By...navinkumardhoprephotography.com

Different data structures are having different applications

Data Structure Applications

- **Linked-List**

- *The linked list is fundamental, and as the simplest persistent, recursive data structure, is used everywhere.*
- *Linked list are usually a basic dynamic data structure which implements **queues and stacks***
- *In **web-browsers**, where it creates a linked list of web-pages visited, so that when you check history (traversal of a list) or press back button, the previous node's data is fetched.*

Data Structure Applications

- **Stack**

- *To **reverse a word**. You push a given word to stack - letter by letter - and then pop letters from the stack.*
- *An "**undo**" mechanism in text editors; this operation is accomplished by keeping all text changes in a stack.*
 - Undo/Redo stacks in Excel or Word.
- ***Language processing** :*
 - compiler's syntax check for matching braces is implemented by using stack.
- *A **stack of plates/books** in a cupboard.*
- *A **garage that is only one car wide**. To remove the first car in we have to take out all the other cars in after it.*
- *Wearing/Removing **Bangles**.*
- ***Back/Forward** stacks on browsers.*
- *Support for **recursion***
 - Activation records of method calls.

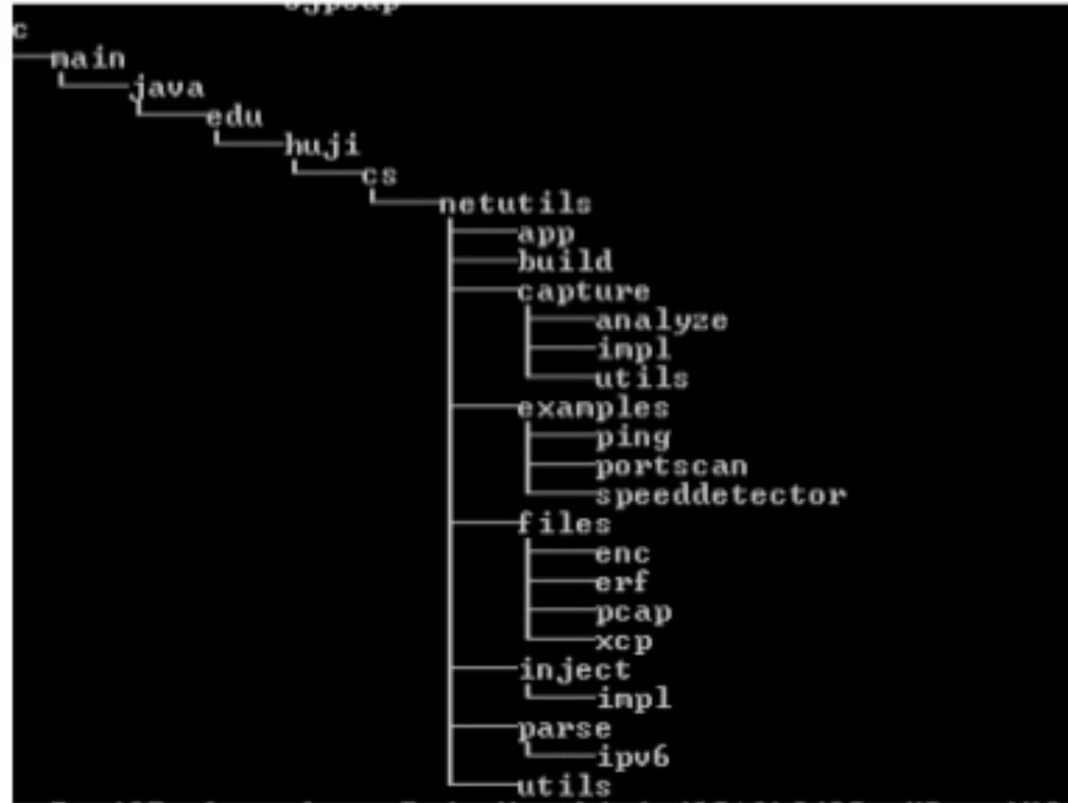
Data Structure Applications

- Queue



Data Structure Applications

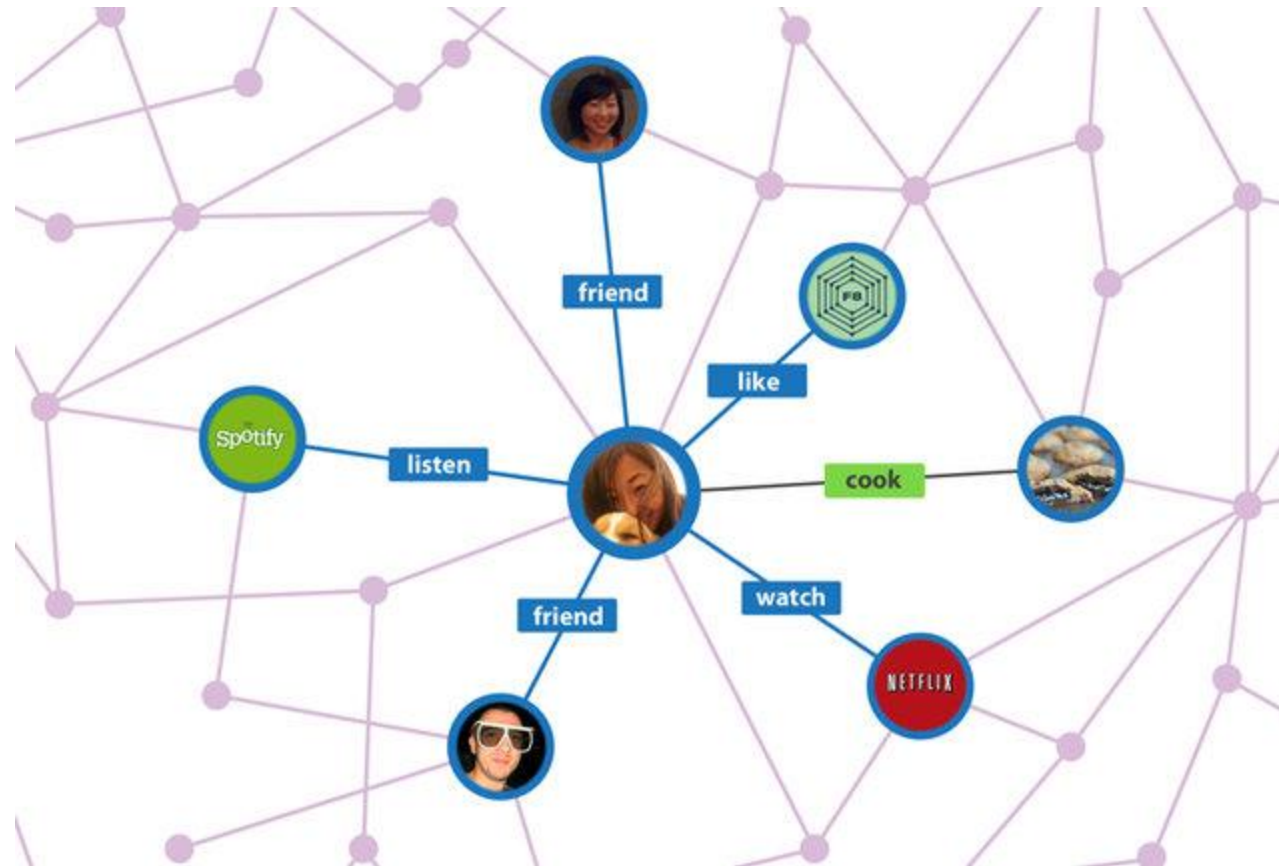
- Tree



Folders in Operating System

Data Structure Applications

- Graph



Thank You